

Database Options

Now we are ready to start working with databases. Laravel supports multiple database systems out of the box. You can use MySQL, PostgreSQL, SQLite, or SQL Server. You can also use in-memory databases for testing.

The great thing about Laravel is that it provides a consistent API for working with different databases. You can switch between databases without changing your code. This is possible because Laravel uses the Query Builder and Eloquent ORM to interact with databases. We also use migrations to create and manage database schemas.

.env File

Laravel uses the `.env` file to store configuration settings. This file stores things like database credentials, application keys, and more. You can find the `.env` file in the root of your Laravel project. It is a hidden file, so you might need to show hidden files to see it.

Let's talk a little bit about the common options for databases in Laravel.

SQLite

Depending on how you setup Laravel, you may have already been asked which database you want to use. If not, Laravel uses SQLite by default. SQLite is a lightweight database that stores data in a single file. It is great for development and testing because it requires no configuration. You can use SQLite for small to medium-sized applications.

If you're building a personal blog or something like that, SQLite is a good choice. You can easily switch to a different database later if needed. If you open up the `.env` file, you will probably see the following:

```
DB_CONNECTION=sqlite
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=laravel
# DB_USERNAME=root
# DB_PASSWORD=
```

This tells Laravel to use SQLite as the database. The other database configurations are commented out. If you're using SQLite, you don't need to worry about those settings.

MySQL

Next up is MySQL, which is a popular open-source relational database management system. It is widely used in web development. If you're building a large application or working with a team, you might want to use MySQL. It is fast, reliable, and scalable and is often used with PHP applications.

If you want to use MySQL, you need to install it on your machine. You can download MySQL from the [official website](#). You can also use a tool like [MAMP](#) or [XAMPP](#) to install MySQL.

Once you have MySQL installed, you need to update the `.env` file with your database settings. Here is an example configuration:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=your_database_name
DB_USERNAME=your_database_user
DB_PASSWORD=your_database_password
```

You would replace `your_database_name`, `your_database_user`, and `your_database_password` with your actual database name, username, and password.

PostgreSQL

PostgreSQL is another popular open-source relational database management system. It is known for its advanced features and reliability. If you're building a large application that requires advanced database features, you might want to use PostgreSQL. This is what I will be using for the course, but if you want to use MySQL, you can use the instructions above. If you want to use SQLite, you don't need to worry about these settings.

If you want to use PostgreSQL, you need to install it on your machine. You can download PostgreSQL from the [official website](#).

Once you have PostgreSQL installed, you need to update the `.env` file with your database settings. Here is an example configuration:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=your_database_name
DB_USERNAME=your_database_user
DB_PASSWORD=your_database_password
```

You would replace `your_database_name`, `your_database_user`, and `your_database_password` with your actual database name, username, and password.

In the next lesson, we will install PostgreSQL on our machine and configure it in our Laravel application.